

Here is the complete, compiled, and updated production code.

I have applied the ESP32 Core Version 3.x pointer fix (* and ->) to the BLE scanner so it will compile perfectly. The code includes your Smart Timer logic, the 14S/6S battery prefix system, the SBUS safety cuts, and the LED warning patterns.

The Final Code (Marut_Flight_System.ino)

Copy everything below, paste it directly into your Arduino IDE, and hit **Upload**:

C++

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <Preferences.h>

// =====
// ⚙️ TUNING PARAMETERS (ADJUST THESE)
// =====
const int RAPID_SCAN_ATTEMPTS = 5; // Quick scan 5 times per cycle
const int RAPID_SCAN_TIMEOUT = 2; // Time (in seconds) for each quick scan window
const int CYCLE_PAUSE_TIME = 2000; // Time (in milliseconds) to pause between full cycles
const int MAX_FAILED_CYCLES = 3; // Consecutive failed cycles needed to cut SBUS

const unsigned long ADMIN_TIMEOUT = 60000; // 60-second fallback boot window
// =====

// =====
// 🔊 HARDWARE SETUP Pins (Change as needed)
// =====
const int SBUS_SWITCH_PIN = 4; // Pin driving the CD4042BCN / CD4052 chip
const int LED_PIN = 2; // Warning LED Pin
// =====

Preferences preferences;
String authorized_macs = "";

// State Tracking Variables
unsigned long bootTime = 0;
bool isAdminMode = true;
bool deviceConnected = false;
```

```

int failed_cycles = 0;

#define SERVICE_UUID      "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"

// -----
// 📶 PHASE 1: ADMIN MODE BLUETOOTH CALLBACKS
// -----
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
        Serial.println("📶 Phone Connected! (Timeout timer PAUSED indefinitely)");
    }
    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
        Serial.println("📶 Phone Disconnected! Triggering immediate Flight Mode...");
        bootTime = millis() - ADMIN_TIMEOUT; // Forces timer to expire immediately
    }
};

class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue().c_str();
        if (rxValue.length() > 0) {
            rxValue.trim();
            rxValue.toUpperCase();

            // ADD BATTERY (Expects +14S:MAC or +6S:MAC)
            if (rxValue.startsWith("+")) {
                String entry = rxValue.substring(1);
                if (authorized_macs.indexOf(entry) == -1) {
                    authorized_macs += entry + ",";
                    preferences.putString("mac_list", authorized_macs);
                    Serial.println("✅ ADDED TO FLEET: " + entry);
                } else {
                    Serial.println("⚠️ Battery already exists in fleet!");
                }
            }

            // REMOVE BATTERY (Expects -14S:MAC or -6S:MAC)
            else if (rxValue.startsWith("-")) {
                String entry = rxValue.substring(1) + ",";
                if (authorized_macs.indexOf(entry) != -1) {
                    authorized_macs.replace(entry, "");
                }
            }
        }
    }
};

```

```

        preferences.putString("mac_list", authorized_macs);
        Serial.println("🗑️ REMOVED FROM FLEET: " + entry);
    } else {
        Serial.println("⚠️ Battery not found.");
    }
} else {
    Serial.println("❌ Invalid Command. Format: +14S:MAC or +6S:MAC");
}
Serial.println("📡 Current Fleet Config: " + authorized_macs);
}
}
};

// -----
// 🚨 LED WARNING PATTERN FUNCTION
// -----
void triggerLedWarning(int failures) {
    if (failures == 1) {
        digitalWrite(LED_PIN, HIGH); delay(150); digitalWrite(LED_PIN, LOW);
    }
    else if (failures == 2) {
        for (int i = 0; i < 2; i++) {
            digitalWrite(LED_PIN, HIGH); delay(150); digitalWrite(LED_PIN, LOW); delay(150);
        }
    }
    else if (failures >= MAX_FAILED_CYCLES) {
        for (int i = 0; i < 3; i++) {
            digitalWrite(LED_PIN, HIGH); delay(150); digitalWrite(LED_PIN, LOW); delay(150);
        }
        digitalWrite(LED_PIN, HIGH); // Stay SOLID Red
        Serial.println("💡 LED: Solid RED (Critical System Failure)");
    }
}

// -----
// ⚙️ INITIALIZATION
// -----
void setup() {
    Serial.begin(115200);
    delay(2000);

    pinMode(SBUS_SWITCH_PIN, OUTPUT);
    pinMode(LED_PIN, OUTPUT);

```

```

digitalWrite(SBUS_SWITCH_PIN, LOW); // Lock SBUS line tightly on boot
digitalWrite(LED_PIN, LOW);

preferences.begin("marut_app", false);

// Default pre-loaded fleet with new prefix definitions for your existing batteries
authorized_macs = preferences.getString("mac_list",
"14S:D0:D8:9C:8A:89:B4,6S:DE:44:73:F5:43:94,6S:EB:54:0E:3F:68:1A,");
preferences.putString("mac_list", authorized_macs);

Serial.println("\n=====");
Serial.println("🔧 PHASE 1: BOOTING ADMIN PORTAL (60s Window)");
Serial.println("📶 Registered Fleet: " + authorized_macs);
Serial.println("=====");

BLEDevice::init("Marut_Config");
BLEServer *pServer = BLEDevice::createServer();
pServer->setCallbacks(new MyServerCallbacks());

BLEService *pService = pServer->createService(SERVICE_UUID);
BLECharacteristic *pCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE
);

pCharacteristic->setCallbacks(new MyCallbacks());
pService->start();

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
pAdvertising->setScanResponse(true);
BLEDevice::startAdvertising();

bootTime = millis();
}

// -----
// 🔄 CORE LOOP & FLIGHT RUNTIME
// -----
void loop() {
    // 1. Handle Admin Mode Timeout Closure

```

```

if (isAdminMode && !deviceConnected && (millis() - bootTime > ADMIN_TIMEOUT)) {
    Serial.println("\n🕒 Boot window closed. Transitioning to Flight Security Scanner...");
    BLEDevice::deinit(true); // Terminate peripheral architecture to clean RAM
    isAdminMode = false;

```

```

    // Spin up the Core Scanner Engine
    BLEDevice::init("");
    Serial.println("🔒 Switched to Flight Scanner Mode. Monitoring Power System...");
}

```

```

// 2. PHASE 2: ACTIVE FLIGHT SECURITY SCANNING

```

```

if (!isAdminMode) {
    Serial.println("\n=====");
    Serial.println("🚀 STARTING NEW RAPID SCAN CYCLE");
    Serial.println("=====");

```

```

    int found14SCount = 0;
    int found6SCount = 0;
    String detectedThisCycle = ""; // Prevent double-counting a single battery

```

```

    BLEScan* pBLEScan = BLEDevice::getScan();
    pBLEScan->setActiveScan(true);
    pBLEScan->setInterval(100);
    pBLEScan->setWindow(99);

```

```

    // Execute the rapid scanning phase
    for (int i = 1; i <= RAPID_SCAN_ATTEMPTS; i++) {
        Serial.printf(" [Rapid Scan %d/%d] Scanning airwaves...\n", i, RAPID_SCAN_ATTEMPTS);

```

```

        // --- THE VERSION 3.X FIX IS APPLIED HERE ---

```

```

        BLEScanResults* foundDevices = pBLEScan->start(RAPID_SCAN_TIMEOUT, false);
        int deviceCount = foundDevices->getCount();

```

```

        for (int d = 0; d < deviceCount; d++) {
            BLEAdvertisedDevice device = foundDevices->getDevice(d);
            String currentMac = device.getAddress().toString().c_str();
            currentMac.toUpperCase();

```

```

            // Check if we haven't processed this MAC yet in this specific cycle

```

```

            if (detectedThisCycle.indexOf(currentMac) == -1) {
                String match14S = "14S:" + currentMac;
                String match6S = "6S:" + currentMac;

```

```

        // Evaluate against Authorized 14S criteria
        if (authorized_macs.indexOf(match14S) != -1) {
            found14SCount++;
            detectedThisCycle += currentMac + ",";
            Serial.println(" 🟢 Found Authorized 14S Power System: [" + currentMac + "]");
        }
        // Evaluate against Authorized 6S criteria
        else if (authorized_macs.indexOf(match6S) != -1) {
            found6SCount++;
            detectedThisCycle += currentMac + ",";
            Serial.println(" 🟢 Found Authorized 6S Module: [" + currentMac + "]");
        }
    }
}

pBLEScan->clearResults(); // Crucial step to eliminate ESP32 memory leaks

// Optimization: Break out early if valid configuration limits are met
if (found14SCount >= 1 || found6SCount >= 2) {
    break;
}
delay(500);
}

// --- CYCLE EVALUATION METRICS ---
if (found14SCount >= 1 || found6SCount >= 2) {
    Serial.println("\n✅ SUCCESS: Hardware verification complete.");
    failed_cycles = 0;
    digitalWrite(LED_PIN, LOW); // Structural clear of warning LED

    if (digitalRead(SBUS_SWITCH_PIN) == LOW) {
        Serial.println(" 🔓 HARDWARE UNLOCKED: Connecting RC Receiver line to Flight Controller.");
    }
    digitalWrite(SBUS_SWITCH_PIN, HIGH); // Force Select Line High (Pass RC Signal)
}
else {
    failed_cycles++;
    Serial.printf("\n⚠️ WARNING: Power system validation failed! (Consecutive failures: %d/%d)\n",
failed_cycles, MAX_FAILED_CYCLES);

    triggerLedWarning(failed_cycles);
}

```

```
// --- THE CRITICAL HARDWARE ISOLATION INTERACTION ---  
if (failed_cycles >= MAX_FAILED_CYCLES) {  
    Serial.println("\n❌ CRITICAL: Unverified power payload signature detected!");  
    Serial.println("🔴 TRIGGERING HARDWARE SBUS DISCONNECT 🔴");  
    digitalWrite(SBUS_SWITCH_PIN, LOW); // Dump line to ground to invoke FC Failsafe  
}  
}  
  
// Cycle Pacing Intermission  
delay(CYCLE_PAUSE_TIME);  
}  
}
```